AI 编程实战训练营 · 第二讲

用 Codex 做完一个真实项目

想清楚 → 立规矩 → 搭骨架 → 做 → 把关 → 验收



承接第一讲

这一讲，我们真刀真枪做个项目

第一讲你学了认知、装好了 Codex、会下指令了。今天不再讲零散招式——我们跟着做完一个真实项目，把招式串起来打。走完这一遍，你就真正知道“平时该怎么用 Codex”了。

今天你会看到一套“固定套路”

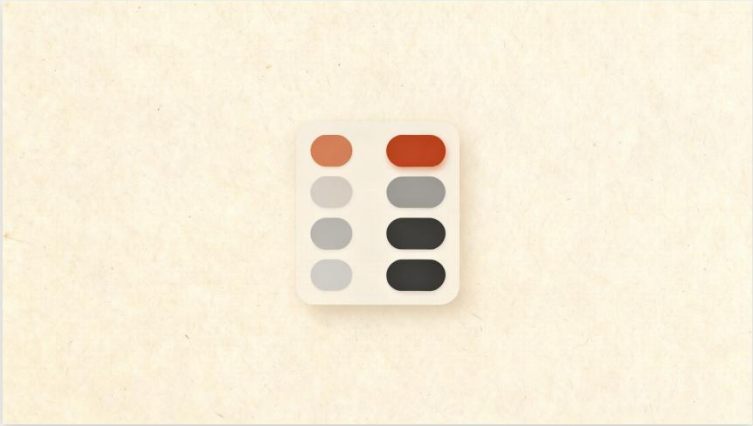
先想清楚 → 用 AGENTS.md 立规矩 → 让 Codex 搭骨架 → 做功能（边做边审批把关、改坏了能还原）→ 迭代 → 验收。以后每个项目，你都照这套走。

今天的项目

一个“客户跟进”小工具

做什么： 一个单页网页，记录每个客户的跟进状态，不再漏跟。

能干嘛： 新增客户（名字+状态+下次跟进）、按状态筛选、标记完成，数据存在本地。



前提（一句话）： 假设你已经按第一讲装好并登录了 Codex、网络也通了。今天不再讲环境配置，直接进入“怎么用”。

动手前

先想清楚，再立规矩

目标文档 · AGENTS.md

第一步 · 想清楚

把想法收敛成“目标文档”

北极星：随手记下每个客户的跟进状态，不漏跟。

给谁用：我自己 / 小团队，每天点几下。

只做（≤3）：加客户、按状态筛选、标记完成。不做：登录、云同步、推送。

验收：加一条→出现在列表；筛选正确；刷新后还在。



为什么先想清楚？AI 执行极快，方向错了它会“高效地”把错的做完。花 5 分钟想清楚，省几小时返工。

第二步 · 立规矩

AGENTS.md：给 AI 的“入职手册”

AGENTS.md 是放在项目里的一份说明书。Codex 每次开工前会先读它，然后自动按里面的约定办事——你不用每次重复交代“用中文、别加多余功能”。



放哪：项目根目录，文件名就叫 AGENTS.md，它自动读取。

好处：约定一次写好，长期生效——你不用每次重复叮嘱它。

实操 · 给这个项目立规矩

我们的 AGENTS.md 长这样

```
# 这是什么项目

一个客户跟进单页网页，自己/小团队用。


# 约定（每次都照做）

· 文字/注释/界面都用中文
· 纯前端单文件，数据存 localStorage
· 改动前先说一句计划


# NEVER（绝不要）

· 不加没要求的功能（登录/云同步/推送）
```

怎么建？ 不用手写——直接对 Codex 说“帮我在项目根目录建一个 AGENTS.md，内容包含以下约定……”，它建好、你审批即可。

AGENTS.md · 两条铁律

只放“稳定的”，写完要“验证”

① 只放稳定铁律

它每次开工被“注入”到 AI 的记忆里，所以必须精炼。放“文字用中文”这种长期规则；别放“现在做到第 2 个功能”这种会变的状态（那属于进度、进接力手册）。

② 写完要验证

别以为写了就生效。开个新对话派个活，看它是否遵守（如“加个页脚”→文字是中文、且先说了计划）；让它做 NEVER 里禁止的事，它应停下提醒你。

一句话：AGENTS.md 放“永远为真的规矩”；进度状态放别处。写了不等于生效，亲自验证。

动手

搭骨架 → 做功能 → 把关

让 Codex 干活，你审批与验收

第三步 · 搭骨架

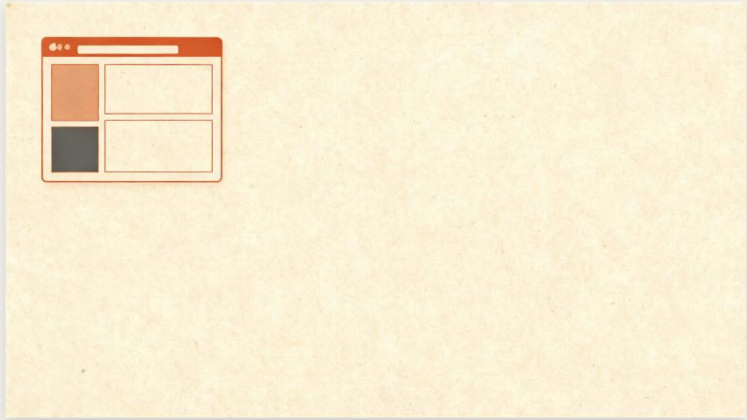
让 Codex 先搭好项目“骨架”

想清楚、立好规矩后，别急着做功能，先让 Codex 一次建好几份基础文件：

docs/目标.md — 北极星与验收落盘

AGENTS.md — 刚立的规矩

验收清单.md — “做完长什么样”的可勾选项



机制：骨架 = 项目的“外部大脑”。 AI 会失忆、你会忘、人会换——把北极星、规矩、验收写成文件，项目就不依赖某次对话，换会话、换人都接得住。

第四步 · 做第一个功能

好指令 → 计划 → 预览 → 批准

你： 按 AGENTS.md 做 index.html：一个“新增客户”表单（名字+状态+下次跟进），提交后加到下面列表，数据存 localStorage。先说计划再动手。

Codex： 计划——表单+列表，提交写入 localStorage、加载回显；纯前端不引框架（符合约定）。

—— 改动预览 · index.html（新建，整片绿色）—— 〔批准〕

你： 批准 → 填“张总/待跟进/周五”→ 出现在列表 → 刷新还在。通过。

它先给计划（可拦下错思路）、再给预览（先审后批）、落地后你亲自核实——别信断言。

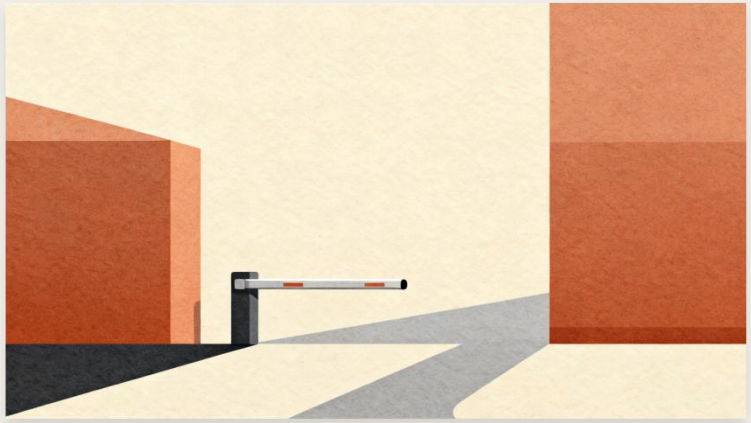
边做边把关 · 审批模式

它动手前，你先点头

只读：只出主意不动手。

自动（默认）：项目文件夹内随便改，出界或联网才问你。

完全访问：读任意文件、联网，谨慎临时用。



机制：沙箱把它圈在项目文件夹里。想碰文件夹外或联网会被拦下问你，这道围栏是操作系统级强制的（Mac 上叫 Seatbelt）——物理护栏，不是口头约定。放胆用，它闯不出这个圈。

改坏了怎么办 · 四层还原

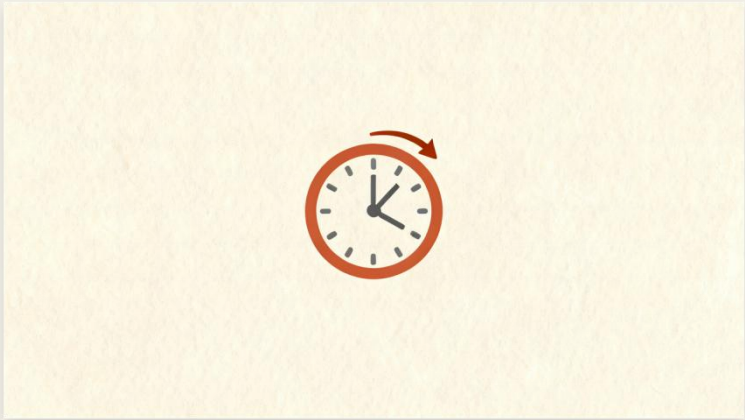
放胆试，救得回

① **拒绝** — 预览时发现不对，直接拒绝。

② **撤销单次** — 已落地的某次改动，还原回去。

③ **回滚检查点** — 一句“回到还原点”，整个项目退回。

④ **隔离丢弃** — 大胆试验用“影分身”，失败整个丢掉。



黄金习惯：大改之前先对它说一句“先存个还原点”。这一秒，能在灾难时省你几小时。

第五步 · 迭代

加功能：按状态筛选

你： 加一个“按状态筛选”的下拉（全部 / 待跟进 / 已完成）；再给每条加“标记完成”按钮，点了把状态改成已完成。其余别动。

Codex： —— 改动预览（有红有绿，只动列表渲染和筛选逻辑）—— 〔批准〕

你： 批准 → 选“待跟进”只显示待跟进 → 点“标记完成”后它从待跟进里消失。通过。

读红绿：改动小而集中、只动该动的地方，就对了；一个小需求却给你一大片红绿，警惕它理解偏了。每加一块，回头看一眼北极星。

第六步 · 验收

让 Codex 对照清单跑一遍

每做完一块，别凭感觉说“好了”。让它对照验收清单逐条自检：

你： 对照 验收清单.md 逐条检查现在的状态，每项告诉我通过还是没通过。

Codex： 加一条→出现在列表　筛选正确　刷新后数据还在



全绿才算“做完了”，而不是凭感觉——这也是“别信断言”。修好一个 bug，就往清单加一条对应检查。

收尾 · 换人也不断片

给项目留一份“交接说明”

接力手册

一份文件写清：项目是什么、北极星、怎么跑起来、现在做到哪、下一步。让完全没接触的人/新对话读它就能接着干。

坑池

踩坑当天记：现象 / 根因 / 修法 / 适用条件。同一个坑，记下来下次 5 秒解决，不记下次再花一小时。

这两份都让 Codex 帮你建和维护。收尾四件套：跑验证 → 存还原点 → 更新接力手册 → 新坑入池。

记住这套“固定套路”

以后每个项目，都照这条线走



北极星在最前、验收在最后；中间边做边审批、随时能还原。这就是你以后用 Codex 的日常节奏。

术语

AI 词汇扫盲

Agent · API · Tool · MCP · Skill

术语 · 关系全景

它们怎么套在一起

1	API	最底层：两个程序之间“怎么对话”的约定
2	Tool	把一个能力（常是一个 API）包成“给模型用的工具”
3	MCP	一套标准，让工具即插即用地接给 Agent
4	Agent	模型（大脑）+ 工具（手脚）+ 循环，自己把任务做完
5	Skill	打包好的“做事流程”，按需喂给 Agent，让它更专业

记忆：Agent 是工人，Tool 是它的工具，MCP 是工具的插座，Skill 是施工手册，API 是底层的砖。

术语 · API（接口）

API：程序间的“调用约定”

两个软件之间“按格式问答”的通道——一个程序用它调用另一个程序的能力或数据。

是什么

一套请求–响应的约定：地址、参数格式、返回格式。

起什么作用

让你用上别家的能力：天气、地图、支付、登录……

怎么组成

接口地址 + 输入参数规范 + 输出结果规范。

边界在哪

只是通道和约定，本身不含智能——规定“怎么问怎么答”，不决定“要不要问”。

术语 · Tool（工具）

Tool：给模型用的“一个能力”

模型可以决定“调用这个工具”，去做自己光靠“说”做不到的事。

是什么

一个函数 / 能力 + 一段给模型看的说明书。

起什么作用

把模型从“只会说”扩展到“能做”：查库、发信、跑代码、搜网页。

怎么组成

能力本体（底层常是包了个 API）+ 名称 / 用途 / 参数说明。

边界在哪

模型只能用你给它的工具；每个工具能力单一；放不放行由审批 / 权限管。

术语 · Agent（智能体 / 代理）

Agent：会自己动手的 AI

大脑（大模型）+ 手脚（工具 + 循环）——能自己规划、调用工具、一步步把任务做完。

是什么

模型 + 工具集 + 循环控制 + 上下文记忆。

起什么作用

让 AI 不只回答，而是“动手把事做完”。Codex、Claude Code 都是 Agent。

怎么组成

读上下文 → 规划 → 调工具 →（审批）→ 看结果 → 再想。

边界在哪

能力取决于给它的工具和权限；受上下文窗口限制；会幻觉，要审批 + 验收。

术语 · MCP（模型上下文协议）

MCP：工具的“USB 标准”

一套连接标准，让任何 Agent 用统一方式，即插即用地接入外部工具和数据。

是什么

一套连接协议 + MCP 服务器（对外暴露一批工具 / 数据）。

起什么作用

不用为每个工具单独定制对接，插上就能用。

怎么组成

MCP 服务器（提供工具）+ 客户端（Agent 按协议调用）。

边界在哪

它是“连接标准”，本身不产生智能；接什么、给多大权限由你控制。

术语 · Skill（技能）

Skill：打包好的“做事流程”

把某类事的流程和经验打包，按需喂给 Agent，让它在特定任务上更专业。

是什么

触发说明（何时用）+ 操作步骤 / 规则 + 附带脚本 / 模板 / 资料。

起什么作用

高频、专业的工作流固化，遇到相关任务就调用，不用从头教。

怎么组成

说明 + 步骤 + 资源文件（如“做 PPT”“处理 PDF”的技能）。

边界在哪

只在相关任务触发时加载；是“经验封装”，调度已有能力，不凭空造新能力。

进阶功能 · 先知道有，用到再学

Codex 还能这些

切换模型/思考强度

难题想得深，简单活跑得快

发图 / 生成图

截图发给它、让它出图

本地代码审查

定稿前多一双眼睛

联网搜索 / 云端任务

查最新、不占本机

子代理

并行拆活更快

MCP / 自动化

接外部工具、一键或定时跑

别为学而学——遇到瓶颈了，再回来解锁对应的这一项。

今天的收获

你会用 Codex 做项目了

- ✓ **一套固定套路** — 想清楚 → 立规矩 → 搭骨架 → 做 → 把关 → 验收 → 收尾
- ✓ **AGENTS.md 立规矩** — 把约定一次写好，AI 每次自动遵守；只放稳定的、写完验证
- ✓ **边做边把关** — 审批模式 + 沙箱围栏 + 四层还原，放胆试也救得回
- ✓ **术语不再懵** — Agent / API / Tool / MCP / Skill 分别是什么

以后接任何一个小项目，你都知道第一步先干嘛、每一步该怎么把关。

动手练习

照今天这套流程，做你自己的一个小工具



先写目标文档和 AGENTS.md，再让 Codex 搭骨架、做第一个功能，最后让它对照验收清单自检一遍。

下一讲 · 从想法到项目 + 不怕断片